Association for
Computing Machinery
INSAT Student Chapter

Data structures & STL

Presented by : Med Aziz chaibi & Med Nadhir Djebbi

What is STL?

The Standard Template Library (STL) in C++ provides a powerful suite of data structures and algorithms that can significantly improve the efficiency and organization of your code. Understanding these data structures is fundamental for writing clean, efficient, and maintainable programs in C++

Advantages of Using STL

Efficiency: STL containers and algorithms are highly optimized.

Reusability: The templated nature of STL allows you to use these structures with any data type.

Ease of Use: STL reduces the need for manual memory management, making your code cleaner and less error-prone.

Flexibility: Different containers can be swapped with minimal changes to the code.

Association for
Computing Machinery
INSAT Student Chapter

Data Structures

A data structure is a way to organize and store data in a computer so it can be used efficiently. Each type helps solve specific problems by offering different methods to access and manipulate the data.

Dynamic arrays : Vectors

Association for Computing Machinery INSAT Student Chapter

A dynamic array is an array whose size can be changed during the execution of the program. The most popular dynamic array in C++ is the vector structure, which can be used almost like an ordinary array.

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    /*Create a vector
      vector<Data_type> Name(Optional:size,Optional:val);
    */
    vector<int> Vecint;//Empty Vector of ints
    vector<long long> VecLong(10);//vector of size 10
    vector<string> VecString(2,"ACM");//vector of size 2 ["ACM","ACM"]

}
```

Operations on vectors

Association for Computing Machinery INSAT Student Chapter

```
#include<bits/stdc++.h>
using namespace std;
int main() {
    vector<int> v;
    v.push_back(1);// adds 1 to the end of the vector v={1} Complexity: O(1)
    v.push_back(2);// v={1,2}
    v.push_back(3);// v={1,2,3}
    v.pop_back();// removes the last element in the vector v={1,2} Complexity: O(1)
    int a=v.back()// returns the value of the last element in v,a=2 Complexity: O(1)
    int b=v.size()// returns the number of elements in v ,b=2
    int c=v[0];// 1 (0-indexed) Complexity: O(1)
}
```

Operations on vectors

```
v.begin()//used to return an iterator pointing to the first element of the vector container
v.end()//is used to return an iterator pointing next to last element of the vector container
```

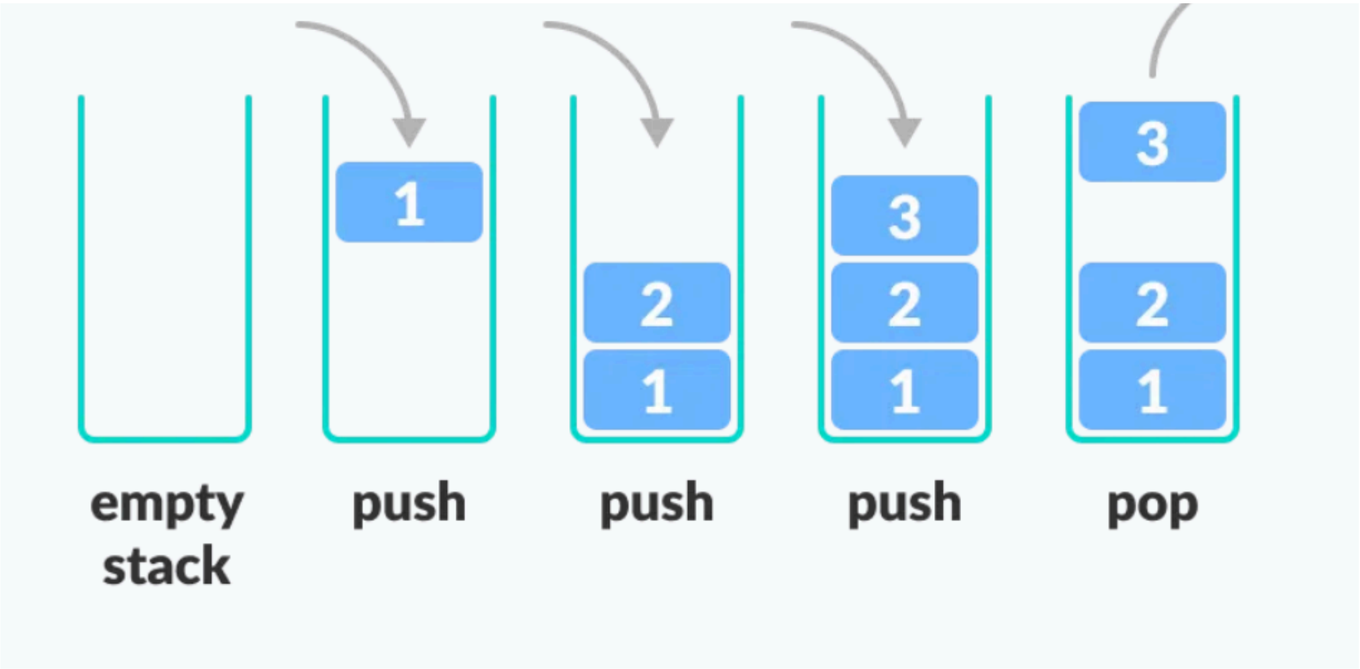
```
vector<int> Vec;
for (int i=1; i<=15; i++) Vec.push_back(i);
// erase the 6th element
Vec.erase (Vec.begin()+5);

// erase the first 3 elements:
Vec.erase (Vec.begin(),Vec.begin()+3);
```

Stack

Association for Computing Machinery INSAT Student Chapter

A stack is a Last In First Out (LIFO) data structure that supports three operations, all in O(1) time. Think of it like a real-world stack of papers.



Operations on Stacks

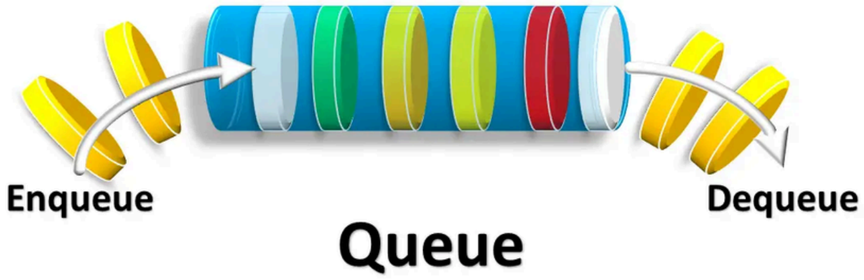
```
#include<bits/stdc++.h>
using namespace std;
int main(){
    /*
      Create a stack
      stack<Data_type> Name;
    */
    stack<int> s;//creating a stack of integers
    s.push(1);//adds 1 to s, s=[1];
    s.push(2);//adds 2 to s,s=[1,2];
    s.pop();//remove the last added element from the stack,s=[1];
    cout<<s.empty();//returns 1 if the stack is empty or 0 if the stack contains at least one element
    cout<<s.top();//returns the element in the top of the stack
}
```

Queue

Association for Computing Machinery INSAT Student Chapter

A queue is an abstract data structure that contains a collection of elements. Queue implements the FIFO mechanism , the element that is inserted first is also deleted first. In other words, the least recently added element is removed first in a queue.

FIFO (First in First Out)



Operations on queues

Association for Computing Machinery INSAT Student Chapter

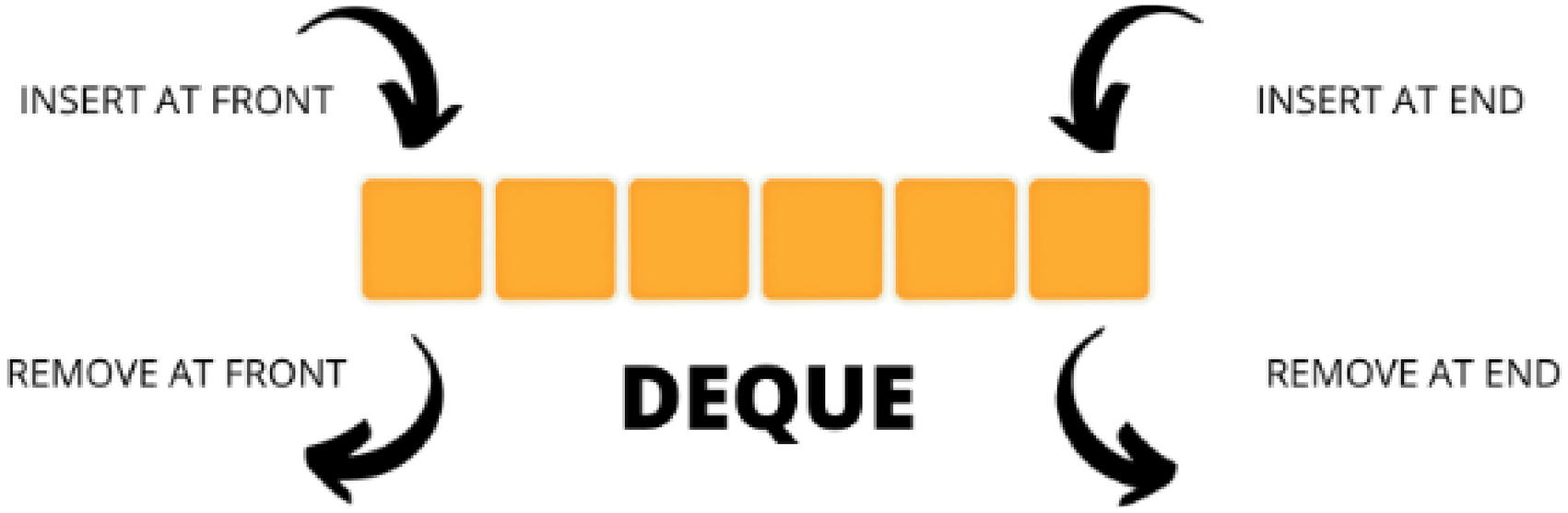
```
#include<bits/stdc++.h>
using namespace std;
int main(){
    /*
      Create a queue
      queue<Data_type> Name;
      The Complexity of all the operations is O(1)
    */
    queue<int> q;//creating a queue of integers
    q.push(1);//adding 1 to the back of the queue q={1};
    q.push(2);//q={1,2}
    q.pop();//removes the first element from the queue q={2};
    cout<<q.front();//returns the first element of the queue; 2
    cout<<q.empty();//returns 1 if the queue is empty  else it returns 0;
}
```

Deque

Association for Computing Machinery INSAT Student Chapter

A deque (double-ended queue) is a data structure that allows elements to be added or removed from both the front and the back, providing greater flexibility than a regular queue. It supports efficient insertion and deletion operations at both ends

All operations have a complexity of O(1) .



Operations on deques

Association for Computing Machinery INSAT Student Chapter

```
#include <bits/stdc++.h>
using namespace std ;
int main ()
{
    deque <char> d ; // Create a deque of characters
    q.push_front('a'); // ADD 'a'to the front d={'a'}
    q.push_back('b'); // ADD 'b' to the back d={'a','b'}
    q.push_front('c'); // d={'c','a','b'}
    cout << d[2] ; // Output the element indexed 2 (b)
    d.pop_front() ;// Remove the first element d={'a','b'}
    d.pop_back() ;// Remove the last element d={'a'}
    cout << d.size() ; // Output the deque size (1)
    return 0;
}
```

Pairs

Association for Computing Machinery INSAT Student Chapter

A pair in C++ is described as a container that combines two elements of the same or different data types. Pair in C++ consists of two elements, first and second (must be in this order), and they are accessed using the dot (.) operator and the keyword first or second

Operations on pairs

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    vector<pair<int,string>> VP; //Create an array of pairs
    VP.push_back({18,"Tres Bien"});//ADD a pair
    VP.push_back({15,"Bien"});
    VP.push_back({12,"Assez Bien"});

    for(auto i:VP)
        cout << i.first << i.second<<endl ;
    // 18 Tres Bien
    // 15 Bien
    // 12 Assez Bien
    return 0;
}
```

Priority Queue

Association for Computing Machinery INSAT Student Chapter

A priority queue is a data structure where each element has a priority, and elements with higher priority are served before those with lower priority. In case of equal priority, elements are served based on their order in the queue.

```
#include <bits/stdc++.h>
using namespace std ;
int main ()
{
    priority_queue <int> PQ; // Create a priority queue of integers
    PQ.push(2); // ADD 9 to the priority queue PQ={2}  Complexity (O(logn))
    PQ.push(9); // PQ={9,2}
    PQ.push(5) // PQ={9,5,2}
    PQ.pop() // Removes the first item PQ={5,1}          Complexity O(logn)
    cout << PQ.top() ; // Output the first element (9) Complexity O(1)
    return 0;
}
```

Association for Computing Machinery INSAT Student Chapter

Set

A set in C++ stores unique elements that are automatically sorted in ascending order. Duplicate values are ignored, and while elements can be added or removed, their values cannot be modified once inserted. Since the order of elements is determined by sorting, sets do not support access by index numbers.

Operations on sets

Association for Computing Machinery INSAT Student Chapter

```
set<string> S ; //Create a set of strings
S.insert("ACM"); //S={"ACM"}      O(logn)
S.insert("ACM") // S={"ACM"}
S.insert("INSAT") // S={"ACM","INSAT"}
S.count("INSAT") // 1              O(logn)
cout<<S.count("MPI") ; //0
S.erase("ACM") //S={"INSAT"}      O(logn)
```

```
set<char> S ={'z','a','s','z','b'};

cout << S.size() ; //4   O(1)

for(auto i:S)
    cout << i; // a b s z
```

There are more types of sets :

1. Unordered Set (unordered_set<>) O(1)

A collection of unique items where the order does not matter, and you can quickly check if an item is present.

2. Multiset (multiset<>) O(logn)

A collection that can hold duplicate items and keeps them sorted.

3. Unordered Multiset (unordered_multiset<>) O(1)

A collection that can hold duplicate items where the order does not matter, and you can quickly check for items.

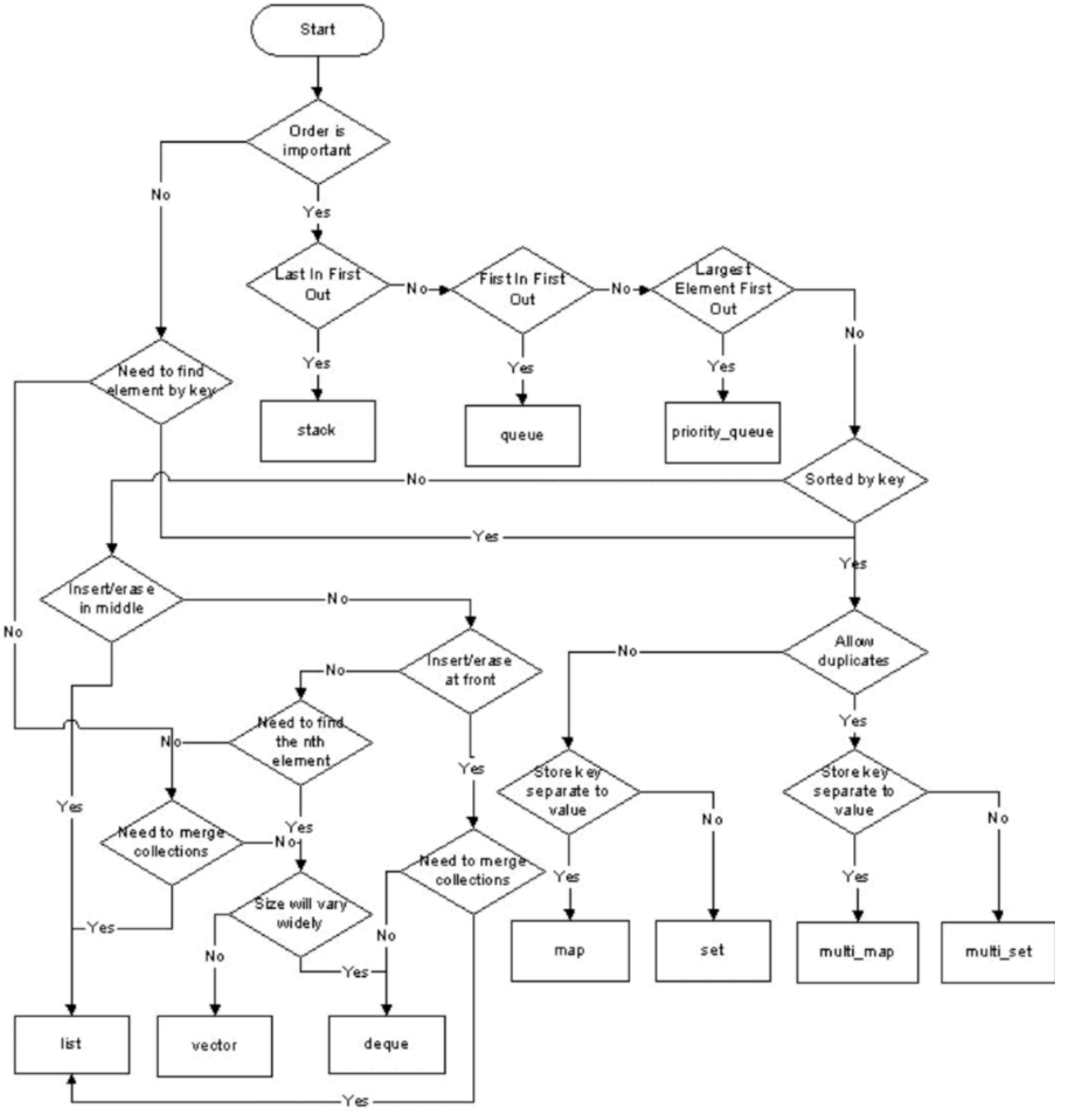
Maps

A map is a generalized array that consists of key-value-pairs. While the keys in an ordinary array are always the consecutive integers 0,1,...,n−1, where n is the size of the array, the keys in a map can be of any data type and they do not have to be consecutive values.

Operations on maps

Association for Computing Machinery INSAT Student Chapter

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
    map <string,int> CLmap ; // Create a map of string keys and int values
    CLmap["Real Madrid"]=15; // ADD the key and its value to the map O(logn)
    CLmap["FC Barcelona"]=5;
    CLmap["FC Bayern"]=6;
    cout<< CLmap["Real Madrid"] ; // 15
    cout<< CLmap["PSG"] ; // The key will be added to the map with a value 0
    cout<< CLmap.count("Real Madrid") ;// Output 1 if the key exists else 0 O(logn)
    for(auto i:CLmap)
        cout << i.first <<" "<< i.second<<'/' ;
    // FC Barcelona 5 / FC Bayern 6 /PSG 0/Real Madrid 15
    return 0;
}
```


Start
Order is important
No
Yes
Last In First Out
No
First In First Out
No
Largest Element First Out
No
Yes
stack
Yes
queue
Yes
priority_queue
Need to find element by key
No
Sorted by key
Yes
Yes
Insert/erase in middle
No
No
Insert/erase at front
No
Allow duplicates
No
Need to find the nth element
No
Yes
Store key separate to value
No
Yes
Store key separate to value
No
Yes
Need to merge collections
Yes
No
Yes
Need to merge collections
Yes
Size will vary widely
No
No
Yes
map
set
multi_map
multi_set
list
vector
deque
Yes

Association for Computing Machinery INSAT Student Chapter

Let’s solve some problems

https://codeforces.com/problemset/problem/4/C

https://codeforces.com/problemset/problem/1703/B

texthttps://shorturl.at/psc20https://shorturl.at/psc20

Association for Computing Machinery INSAT Student Chapter

Numeric Library

The C++ <numeric> library includes common mathematical functions and types, as well as optimized numeric arrays and support for random number generation.

Numeric Library

Association for Computing Machinery INSAT Student Chapter

lcm

Returns the least common multiple of two integers O(log(min(a,b))

```
cout << lcm(12,18) ; // 36
```

gcd

Returns the greatest common divisor of two integers O(log(min(a,b))

```
cout << gcd(12,18) ; // 6
```

Numeric Library

Association for Computing Machinery INSAT Student Chapter

accumulate

Returns the sum of all values in the range (first,last) with a given value O(n)

```
int main()
{
    vector <int> v={1,2,3,4,5};
    cout << accumulate(v.begin(),v.end(),0) ; // 15
    cout << accumulate(v.begin(),v.end(),110) ; // 115
    return 0;
}
```

Numeric Library

Association for Computing Machinery INSAT Student Chapter

iota

Assigns a value to the elements of the array in range (first,last) which is incremented each step by 1

```
vector <int> v (5);
iota (v.begin(),v.begin()+5,15);
for(auto i:v)
{
    cout<<i ; // 15 16 17 18 19
}
```

Association for Computing Machinery INSAT Student Chapter

Cmath Library

The <cmath> library in C++ provides a range of mathematical functions that allow for operations such as basic arithmetic, trigonometric calculations, exponentiation, logarithms, and other advanced mathematical computations. It is commonly used for scientific and mathematical tasks requiring precision and efficiency.

Cmath Library

Association for Computing Machinery INSAT Student Chapter

Sqrt

Add a The sqrt() function returns the square root of a number

```
cout<<sqrt(16)<<endl;//4
```

Floor

Rounds x downward, returning the largest integral value that is not greater than x.

```
cout<<floor(2.3)<<endl;//2
cout<<floor(2.9)<<endl;//2
```

Cmath Library

Association for Computing Machinery INSAT Student Chapter

Ceil

ceil() function ireturns the smallest integer that is greater than or equal to x

```
cout<<ceil(2.3)<<endl;//3
cout<<ceil(2.9)<<endl;//3
cout<<ceil(2)<<endl;//2
```

Round

```
cout<<round(2.3)<<endl;//3
cout<<round(2.9)<<endl;//3
cout<<round(2)<<endl;//2
```

Association for Computing Machinery INSAT Student Chapter

Algorithm Library

The <algorithm> Library in C++ provides a collection of functions for performing common operations on data structures, such as sorting, searching, counting, and manipulating elements. These functions operate on ranges of elements, typically using iterators, and are part of the Standard Template Library (STL).

Algorithm Library

Association for Computing Machinery INSAT Student Chapter

Sort O(nlogn)

```
vector <char> v ={'z','f','a','m','k'};
sort(v.begin(),v.end());

for(auto i:v)
    cout <<i ; // a f k m z
```

Reverse O(n)

```
vector <string> v ={"Association ","For ","Computing ",
                    "And ", "Machinary "};
reverse(v.begin(),v.end());

for(auto i:v)
    cout <<i ; // Machinary And Computing For Association
```

Algorithm Library

Association for Computing Machinery INSAT Student Chapter

find O(n)

```
vector <string> v ={"Association ","For ","Computing ",
                    "And ", "Machinary "};
auto it =find(v.begin(),v.end(),"For ");
if(it!=v.end())
    cout << it-v.begin() ; // 1
else
    cout << "Not Found" ;
```

Resources

Association for Computing Machinery INSAT Student Chapter

https://en.cppreference.com/w/

https://www.youtube.com/watch?v=2olsGf6JIkU&ab_channel=CppCon

Association for
Computing Machinery
INSAT Student Chapter

Thank you
for Your Attention